StackGenVis: Alignment of Data, Algorithms, and Models for Stacking Ensemble Learning Using Performance Metrics

Angelos Chatzimparmpas, *Student Member, IEEE*, Rafael M. Martins, *Member, IEEE Computer Society*, Kostiantyn Kucher, *Member, IEEE Computer Society*, and Andreas Kerren, *Senior Member, IEEE*

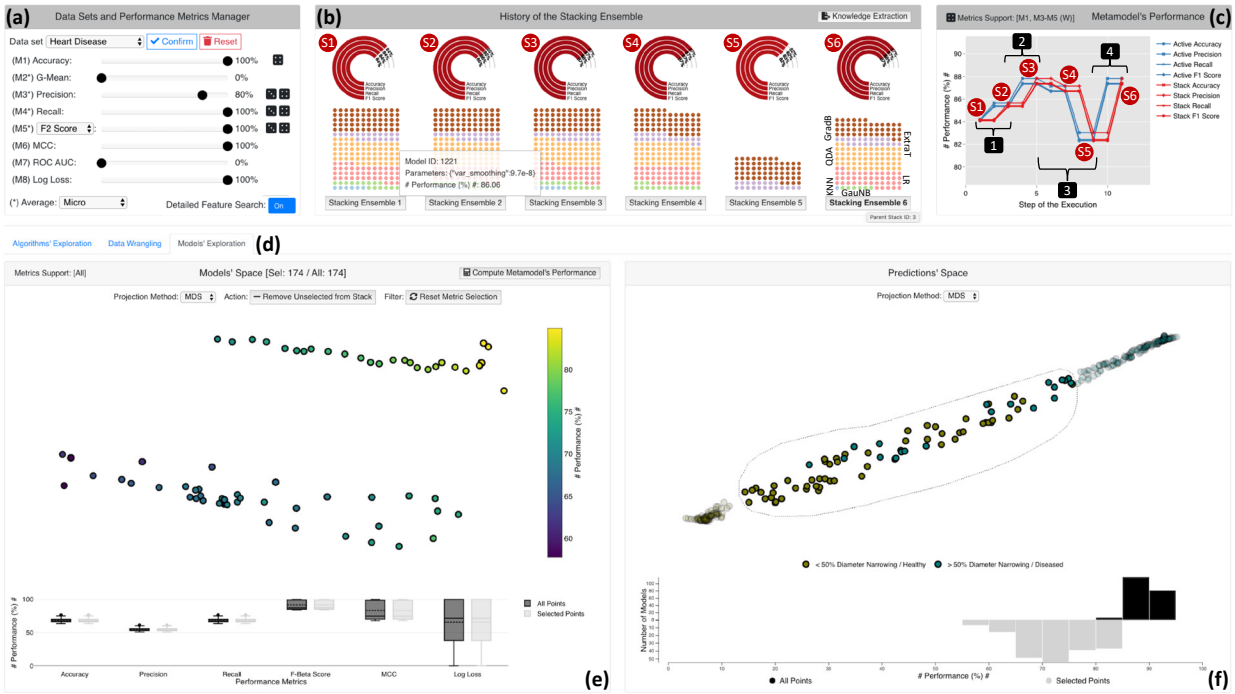

Fig. 1. Constructing performant stacking ensembles from scratch with StackGenVis: (a) a panel for uploading data sets and choosing weights for performance metrics; (b) the history preservation panel with the composition and performance achieved by the user-built stored stacking ensembles; (c) the comparison of the metamodel's performance for both the active and stored stackings, based on four performance metrics (linked to view (a) with a dice glyph showing four); (d) the three exploration modes for the algorithms, data, and models; (e) the projection-based models' space visualization, which summarizes the results of all the selected performance metrics for all models; and (f) the predictions' space visual embedding, which arranges the data instances based on the collective outcome of the models in the current stored stack Ⓢ6 (marked in bold typeface in (b)).

Abstract— In machine learning (ML), ensemble methods—such as bagging, boosting, and stacking—are widely-established approaches that regularly achieve top-notch predictive performance. Stacking (also called "stacked generalization") is an ensemble method that combines heterogeneous base models, arranged in at least one layer, and then employs another metamodel to summarize the predictions of those models. Although it may be a highly-effective approach for increasing the predictive performance of ML, generating a stack of models from scratch can be a cumbersome trial-and-error process. This challenge stems from the enormous space of available solutions, with different sets of data instances and features that could be used for training, several algorithms to choose from, and instantiations of these algorithms using diverse parameters (i.e., models) that perform differently according to various metrics. In this work, we present a knowledge generation model, which supports ensemble learning with the use of visualization, and a visual analytics system for stacked generalization. Our system, StackGenVis, assists users in dynamically adapting performance metrics, managing data instances, selecting the most important features for a given data set, choosing a set of top-performant and diverse algorithms, and measuring the predictive performance. In consequence, our proposed tool helps users to decide between distinct models and to reduce the complexity of the resulting stack by removing overpromising and underperforming models. The applicability and effectiveness of StackGenVis are demonstrated with two use cases: a real-world healthcare data set and a collection of data related to sentiment/stance detection in texts. Finally, the tool has been evaluated through interviews with three ML experts.

Index Terms—Stacking, stacked generalization, ensemble learning, visual analytics, visualization

1 Introduction

Stacking methods (or *stacked generalizations*) refer to a group of ensemble learning methods [45] where several base models are trained and combined into a metamodel with improved predictive power [63].

• *Angelos Chatzimparmpas, Rafael M. Martins, Kostiantyn Kucher, and Andreas Kerren are with Linnaeus University, Växjö, Sweden.*
E-mail: {*angelos.chatzimparmpas, rafael.martins, kostiantyn.kucher, andreas.kerren*}*@lnu.se*

Manuscript received 30 Apr. 2020; revised 31 July 2020; accepted 14 Aug. 2020. Date of publication 13 Oct. 2020; date of current version 15 Jan. 2021. Digital Object Identifier no. 10.1109/TVCG.2020.3030352

In particular, stacked generalization can reduce the bias and decrease the generalization error when compared to the use of single learning algorithms. To accomplish that, stacking enables the blending of different and heterogeneous *algorithms* and their instantiations with particular parameters, i.e., *models*. Other types of ensemble methods are *bagging techniques*, such as random forests (RF) [2], and *boosting techniques*, such as adaptive boosting (AdaB) [16] or gradient boosting (GradB) [7, 21]. A major difference between these ensemble methods is that stacking can use both bagging and boosting techniques in combination with simpler algorithms, stacked in different layers. It uses a *meta-learner* to aggregate the predictions of the last layer and obtain the best performance, which is absent in the other ensemble methods.

In numerous Kaggle competitions [20], stacking ensembles led to award-winning results. But, when studying such ensembles, it is very hard to understand *why* specific instances, features, algorithms, and models were selected instead of others. Indeed, one of the major challenges in stacking is to select the best combinations of algorithms and models when designing a stacking ensemble from scratch. This issue may keep machine learning (ML) practitioners and experts away from working with complex stacking ensemble methods, even though they could arguably reach very high-performance results. One question that arises from the work by Naimi and Balzer [38] is: **(RQ1)** how to build a stacking ensemble for a given problem with a focus on avoiding such trial and error methods, and/or increasing the overall efficiency?

In spite of this challenge of hardly understanding why a specific configuration works [57], predicting the relation of supply-demand [60] and anomaly/bug reports [18, 19] are areas where stacking has been used successfully. Compelling accuracy results [59] were also observed for text data, where stacking is better than alternative techniques such as voting ensembles [50]. Above all, mixtures of stacked models have been deployed to increase the performance of results in medicine [30, 37, 39]. In the case of healthcare-related problems, however, the difficulties of stacking lead to an even worse situation, because interpretability, fairness in decisions, and trustworthiness of ML models are very critical in the medical field [9]. The recent survey by Sagi and Rokach [45] lists the users' ability to understand how to tune the models as one of the important factors for selecting the appropriate ensemble learning method, too. Thus, another open question is: **(RQ2)** how to monitor and control the complete process of training stacking ensembles, while preserving confidence and trust in their predictive results?

Performance metrics, such as precision or f1-score, are typically adopted to validate if the ML results meet the expectations of the experts and the domain [15, 41, 52, 56]. Multiple metrics are important to avoid the dangers of using single metrics, such as accuracy [32, 54], for every data set. However, comparison and selection between multiple performance indicators is not trivial, even for widely used metrics [12, 46]; alternatives such as Matthews correlation coefficient (MCC) might be more informative for some problems [8]. Further open challenges of using advanced metrics are described in the literature [27, 42]. This leads to one further question: **(RQ3)** what performance/validation metrics fit better to a specific data set, and how can they be combined?

Stacking ensemble learning inspired us to focus on each of the three aforementioned questions that represents an open research challenge. In this paper, we present a knowledge generation model for ensemble learning with the use of visualization (derived from Sacha et al. [44]), and instantiate this model as our visual analytics (VA) system for stacked generalization. Our system, called StackGenVis (see Fig. 1), tries to address the three questions described above by supporting the exploratory combination of 11 different ML algorithms and 3,106 individual models using 8 performance metrics with various modes (see the details in Sect. 4). To address those three open research challenges (**RQ1–RQ3**), StackGenVis supports the following **workflow**: (i) the selection of appropriate validation metrics, (ii) the exploration of algorithms, (iii) the data wrangling, (iv) the exploration of models, and (v) an overarching phase, where the resulting stack is traced and the performance of the stored stack is compared to the current active metamodel. In summary, our contributions consist of the following:

- the composition of a knowledge generation model (KGM) specifically adapted for ensemble learning with the use of VA;
- the implementation of a VA system, called StackGenVis, that follows the KGM mentioned above, consists of novel views that treat models and predictions as high-dimensional vectors, and supports the visual exploration of the most performant and most diverse models for the creation of stacking ensembles;
- the applicability of our proposed system with two use cases, using real-world data, that confirm the effectiveness of utilizing multiple validation metrics and comparing stacking ensembles; and
- the discussion of the followed methodology and the outcomes of several expert interviews that indicate encouraging results.

The rest of this paper is organized as follows. In the next section, we discuss the literature related to visualization of ensemble learning. Afterwards, we describe the knowledge generation model for ensemble learning with VA, design goals, and analytical tasks for attaching VA to ensemble learning. Sect. 4 presents the functionalities of the tool and, at the same time, describes the first use case with the goal of improving another stacking ensemble's results for healthcare data. Next, we demonstrate the applicability and usefulness of StackGenVis with our own real-world data set focusing on sentiment/stance in texts. Thereafter in Sect. 6, we discuss the feedback our VA system received during the conducted expert interviews by summarizing the opinions of the experts and the limitations that lead to possible future work for our approach. Finally, Sect. 7 concludes our paper.

2 Related Work

Visualization systems have been developed for the exploration of diverse aspects of bagging, boosting, and further strategies such as "bucket of models". Stacking, however, has so far not received comparable attention by the InfoVis/VA communities: actually, we have not found any literature describing the construction and improvement of stacking ensemble learning with the use of VA. In this section, we briefly discuss previous works on bagging, boosting, and buckets of models, and highlight their differences with StackGenVis in order to substantiate the novelty of our approach.

2.1 Bagging and Boosting

EnsembleLens [65] is a VA system that focuses on the identification of the best combination of models by visualizing their correlation. Specific feature subsets are chosen to train each algorithm—a technique known as *feature bagging*. Then, the results are combined and ranked based on the performance outcomes for anomalous cases. In contrast, our work is not limited to the anomaly detection task, and it focuses on construction of better-performing ensembles by combining multiple algorithms and using appropriate performance metrics.

BEAMES [11] focuses on regression tasks, and it includes four learning algorithms and a model sampling technique. The output of the system is a ranking that helps the user to decide on a model. In our approach, a metamodel automatically chooses well-performing models. BEAMES includes three performance metrics: (a) residual error, (b) mean squared error, and (c) r-squared error, which are specialized for regression problems. Interestingly, the authors suggest as future work that "there are open research questions about how best to compare multiple models directly". In our system, we address this challenge with the exploration of a finite space of solutions, employing 11 algorithms (that can be further expanded). Exploration of feature importance and instances in BEAMES involves a standard table representation and recommendation of best-performing models for specific instances. In StackGenVis, we present different ways for direct manipulation of instances, highlighting hard-to-classify instances. Three separate techniques are incorporated for feature selection, visualized by an aggregated table heatmap view.

iForest [68] is a VA system that uses dimensionality reduction (DR) to summarize the predictions of each instance; other views explain decision paths of an RF. It also highlights the relationship between the features of the data set and the prediction outcomes. For a specific instance, a new DR projection can be used to show which models performed well or not, and why. The goals and challenges addressed

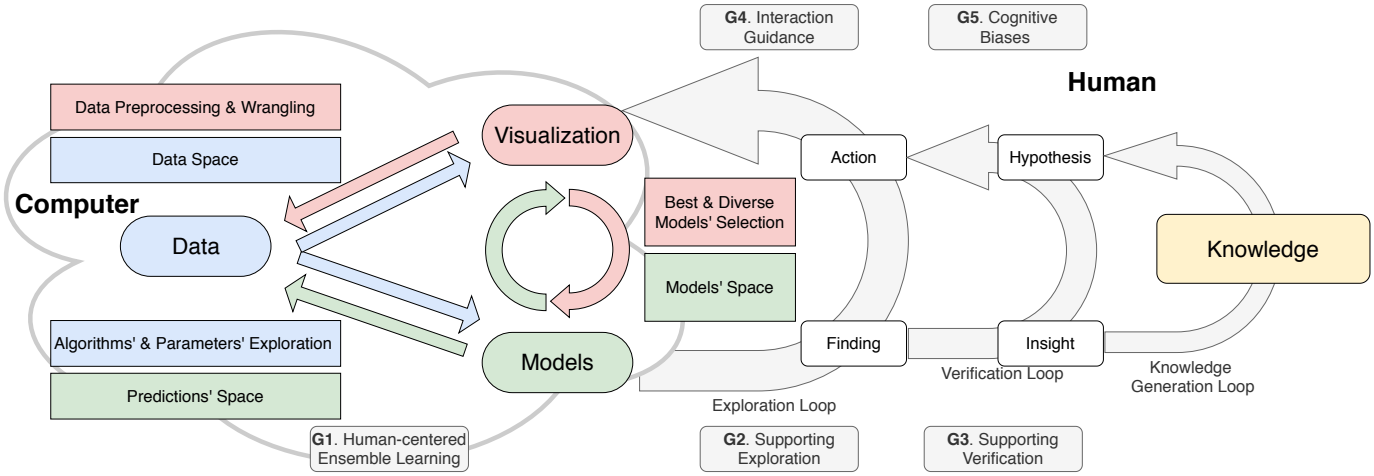

Fig. 2. Knowledge generation model for ensemble learning with VA derived from the model by Sacha et al. [44]. On the left, it illustrates how a VA system can enable the exploration of the data and the models with the use of visualization. On the right, a number of design goals assist the human in the exploration, verification, and knowledge generation for ensemble learning.

by iForest are different than ours: it strives to open the black box of a specific algorithm, while StackGenVis uses a parallel and model-agnostic strategy accompanied by high-level monitoring of the process. Additionally, we utilize multiple validation metrics simultaneously to explore diverse models, instead of relying on decision trees only.

Similarly to iForest, BoostVis [25] also uses DR and other views to compare and improve ML algorithms such as XGBoost [7] or LightGBM [21]. The goal is to diagnose and debug the training process of underperforming trees, which are visualized with trajectories in a DR projection. Our work, in contrast, focuses on the appropriate selection of models to enhance—as much as possible—the prediction power of a stacking ensemble. Moreover, we use three alternative techniques to rank the most important features for several hundreds or thousands of models, and we use multiple performance metrics, with user-defined weightings, to characterize the results.

Schneider et al. [47] employed both bagging and boosting ensembles in an effort to combine the data and model space. The authors applied scatterplots and DR projections for the visualization of the data space, with the goal to add, delete, or replace models from the ensemble model space. Pairs of validation metrics allow the user to select the best models (sorted by performance or similarity). A selection results in an update of the data space. Our approach of aligning the data and model spaces is influenced by this work, but we improved the process by aggregating the alternative performance metric results on top of the projections. Furthermore, we allow the users to define specific weights for each metric and focus on the models that perform well for both the entire data space and specific instances. Finally, StackGenVis does not support direct manipulation of model ensembles [47], as it focuses on exploration of a large solution space before narrowing down to specific well-performing and diverse models.

2.2 Buckets of Models

In a bucket of models, the best model for a specific problem is automatically chosen from a set of available options. This strategy is conceptually different to the ideas of bagging, boosting, and stacking, but still related to ensemble learning. Chen et al. [6] utilize a bucket of latent Dirichlet allocation (LDA) models for combining topics based on criteria such as distinctiveness and coverage of the set of actions performed. Pie charts on top of projections show probability distributions of action classes. Although this work is not similar to StackGenVis in general, we use a gradient color scale to map the performance of each model in the projected space. EnsembleMatrix [55] linearly fuses multiple models with the help of a confusion matrix representation, while supporting comparison and contrasting for model exploration. In our VA system, the user can explore how models perform on each class of the data set, and the performance metrics are instilled into a combined user-driven value. Manifold [66] generates pairs of models and compares them over all classes of a data set, including feature selection. We adopt a similar approach, but instead of comparing a large number of models in a pairwise manner, we aggregate their overall and per-class performance. Then, the user can compare a set of models against the average of all the models before deciding which ones to use.

There is also a group of works that focuses specifically on regression problems [36, 48, 67]. For instance, the more recent tool iFuseML [48] operates with prediction errors in order to present ensemble models with more accurate predictions to the users. The comparison of models is very different in our approach: we use preliminary results from performance metrics in order to select the appropriate models that will boost the final stack performance.

3 Design Goals and Analytical Tasks

In this section, we explain the main design goals that base the development of StackGenVis, together with a knowledge generation model (KGM) for ensemble learning (Fig. 2). Then, we describe the analytical tasks that StackGenVis (and any other VA system) should tackle in order to support the presented KGM with regard to stacking methods.

3.1 Design Goals: Visual Analytics to Support Ensemble Learning

In the following, we define five design goals (**G1–G5**) built on top of the knowledge generation model for VA proposed by Sacha et al. [44]. This original model has two core pillars: the *computer* (Fig. 2, left) and the *human* (Fig. 2, right). On the computer side, the VA system comprises *data*, *visualization(s)*, and *model(s)*. The human side depicts the knowledge generation process, comprising the loops for *exploration*, *verification*, and *knowledge generation*.

Our design goals focus on the knowledge generation in ensemble learning with the use of VA, originating from the analysis of the related work in Sect. 2, our own experiences when developing VA tools for ML (e.g., t-viSNE [5]), and recently conducted literature reviews [3, 4]. We slightly extended the original knowledge generation model for VA [44] to make a better fit for supporting ensemble learning with VA (cf. the description of design goal **G1**) and then aligned our design goals to the different model parts, see the gray boxes in Fig. 2.

G1: Incorporate human-centered approaches for controlling ensemble learning. For our first design goal, we modified the original knowledge generation model for VA [44] by adding components specifically related to ensemble methods [45]. Ensemble learning can be controlled in different ways. Starting from the data, visualization can be used to explore the *data space* (Fig. 2, upper blue arrow) [47]. This

offers new possibilities for direct manipulation of both instances and features. Visualization also enhances the interaction with *data preparation* (Fig. 2, upper red arrow) [25]. Data preprocessing and wrangling benefits from feedback provided by a VA system, for example, in the form of validation metrics that increase the per-model performance of several heterogeneous ML models used in ensemble learning. Next, VA is useful for the exploration and final selection of different *algorithms* that have numerous *parameters* leading to well-performing models (Fig. 2, lower blue arrow) [11]. These models produce predictions that can be stored again as new metadata. If visualized, this *predictions' space* can be manipulated accordingly for raising the overall predictive performance (Fig. 2, lower green arrow). The process of ensemble learning generates a *solution space of models* (Fig. 2, curved green arrow) [47] and more investigations can be done to choose between the *best and most diverse models* of an ensemble (Fig. 2, curved red arrow). The careful design, choice, and arrangement of these aspects and the balance between human-centered vs. automated approaches are essential concepts when developing a VA system [49]. Moreover, the different perspectives of analysts working on a problem can push toward more efficient and effective solutions or receiving results in a shorter amount of time. Synchronous and asynchronous collaboration can empower visualizations dedicated to particular tasks [17]. Building ensembles from scratch by using various ML algorithms might require expert collaboration and intervention, especially when those experts are specialized on individual algorithms. If a VA system supports asynchronous and/or synchronous communication, an individual expert can share his/her knowledge with the others, which could lead to a more desirable outcome.

G2: Support exploration. VA systems enable users to reach crucial *findings* and to take *actions* according to them. This iterative process requires a human-in-the-loop who can thus explore the data and the model through the interactive visualization [1]. As the solution space for ensemble learning is more confusing compared to single ML techniques, keeping track of the history of events and providing provenance for exploring and backtracking of alternative paths is necessary to reach this goal. Furthermore, provenance in VA for ensemble learning increases the interpretability and explainability caused by the complex nature of the method. Although provenance in VA systems has been in the research focus during the past years [40, 43], the work on utilizing analytic provenance [64] is still limited.

G3: Support verification. According to the *insights* gained from the exploration process, users are able to formulate new *hypotheses* that can be efficiently tested with the help of interactive visualization. This goal is valuable especially for ensemble methods, which are harder to train and verify than individual ML models. Annotations within a visualization are used to share insights between analysts or to save information for later use. In storytelling, for example, the annotation is considered as a key element [58]. Keeping notes linked to particular views of a VA system for ensemble learning could be essential for remembering key findings and core actions for reaching good performance results.

G4: Facilitate human interaction and offer guidance. During development of any VA tool, it is key to decide on concrete visual representations and interaction technologies between multiple coordinated views. It is not uncommon to find gaps between visualization design guidelines and their applicability in implemented tools [35], and providing guidance in the complex human-machine analytical process is another challenge [10]. Many different facets are involved in VA for ensemble learning, ranging from diverse ML models and data sets to performance metrics. From a visualization perspective, this heterogeneity leads to multiple views. A careful visual design of the linked views that facilitate human interaction and sophisticated VA systems that guide the user to important aspects will help to disentangle the visual complexity and, in consequence, the cognitive load of the user.

G5: Reveal and reduce cognitive biases. Visualizations should be carefully chosen in order to reduce cognitive biases. Cognitive bias is, in simple terms, a human judgment that drifts away from the actual information that should be conveyed by a visualization, i.e., it "involves a deviation from reality that is predictable and relatively consistent across people" [13]. The use of visualization for ensemble learning could possibly introduce further biases to the already blurry situation based on the different ML models involved. Thus, the thorough selection of both interaction techniques and visual representations that highlight and potentially overcome any cognitive biases is a major step toward realizing this design goal.

3.2 Analytical Tasks for Stacking

To fulfill our design goals specifically in the context of stacking ensemble learning, we have derived five high-level analytical tasks that should be solved by our VA system described in Sect. 4.

T1: Search the solution space for the most suitable algorithms, data, and models for the task. Some of the major challenges of stacking are the choice of the most suitable algorithms and models, the data processing necessary for the selected models, further improvements for the models, and reduction of the complexity of the stack (**G1**). This workflow should be assisted by guidance at different levels, including the selection of proper performance metrics for the particular problem and the comparison of results against the current stack.

T2: Explore the history with all basic actions of the stacking ensemble preserved. There is a large solution space of different learning methods and concrete models which can be combined in a stack. Hence, the identification and selection of particular algorithms and instantiations over the time of exploration is crucial for the the user. One way to manage this is to keep track of the history of each model. Analysts might also want to step back to a specific previous stage in case they reached a dead end in the exploration of algorithms and models (**G2**).

T3: Manage the performance metrics for enhancing trust in the results. Many performance or validation metrics are used in the field of ML. For each data set, there might be a different set of metrics to measure the best-performing stacking. Controlling the process by alternating these metrics and observing their influence in the performance can be an advantage (**G3**).

T4: Compare the results of two stages and receive feedback to guide interaction. To assist the knowledge generation, a comparison between the currently active stack against previously stored versions is important. In general, this includes monitoring the historical process of the stacking ensemble, facilitating interaction and guidance (**G4**).

T5: Inspect the same view with alternative techniques and visualizations. To eventually avoid the appearance of cognitive biases, alternative interaction methods and visual representations of the same data from another perspective should be offered to the user (**G5**).

4 StackGenVis: System Overview and Application

Following our design goals and derived analytical tasks, we implemented StackGenVis, an interactive VA system that allows users to build powerful stacking ensembles from scratch. Our system consists of six main interactive visualization panels (see Fig. 1): (1) performance metric selection (→ **T3**), (2) history monitoring of stackings (→ **T2**), (3) ML algorithm exploration, (4) data wrangling, (5) model exploration (→ **T1** and **T5**), and (6) performance comparison between stacks (→ **T4**). We use the following **workflow** when applying StackGenVis: (i) we choose suitable performance metrics for the data set, which are then used for validation during the entire building process (Fig. 1(a)); (ii) in the next algorithm exploration phase, we compare and choose specific ML algorithms for the ensemble and then proceed with their particular instantiations, i.e., the models; (iii) during the data wrangling phase, we manipulate the instances and features with two different views for each of them; (iv) model exploration allows us to reduce the size of the stacking ensemble, discard any unnecessary models, and observe the predictions of the models collectively (Fig. 1(d)); and (v) we track the history of the previously stored stacking ensembles in Fig. 1(b) and compare their performances against the *active* stacking ensemble—the one not yet stored in the history—in Fig. 1(c).

StackGenVis works with 11 ML algorithms that can be further subdivided into seven separate groups/types: (a) a neighbor classifier (k-nearest neighbor (KNN)), (b) a support vector machine classifier (SVC), (c) a naïve Bayes classifier (Gaussian (GauNB)), (d) a neural network classifier (multilayer perceptron (MLP)), (e) a linear classifier

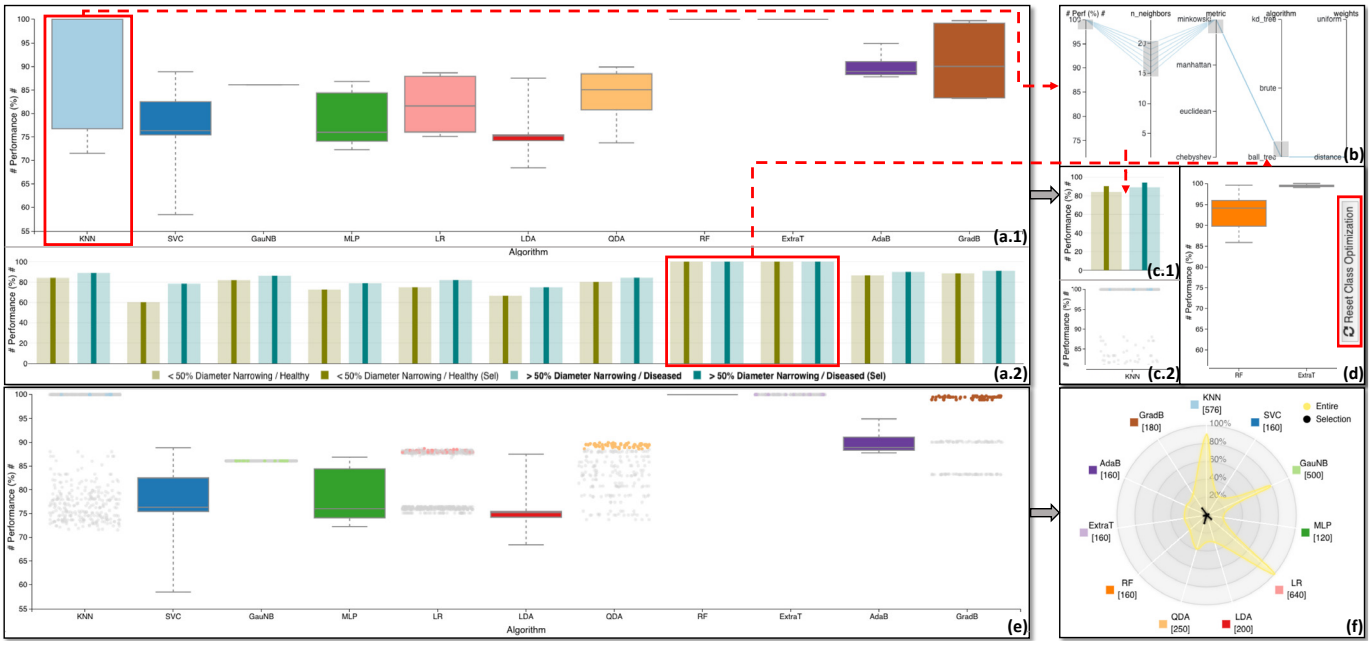

Fig. 3. The exploration process of ML algorithms. View (a.1) summarizes the performance of all *available* algorithms, and (a.2) the per-class performance based on precision, recall, and f1-score for each algorithm. (b) presents a selection of parameters for KNN in order to boost the per-class performance shown in (c.1). (c.2) illustrates in light blue the selected models and in gray the remaining ones. Also from (a.2), both RF and ExtraT performances seem to be equal. However in (d), after resetting class optimization, ExtraT models appear to perform better overall. In view (e), the boxplots were replaced by point clouds that represent the individual models of activated algorithms. The color encoding is the same as for the algorithms, but unselected models are greyed out. Finally, the radar chart in (f) displays a portion of the models' space in black that will be used to create the initial stack against the entire exploration space in yellow. The chart axes are normalized from 0 to 100%.

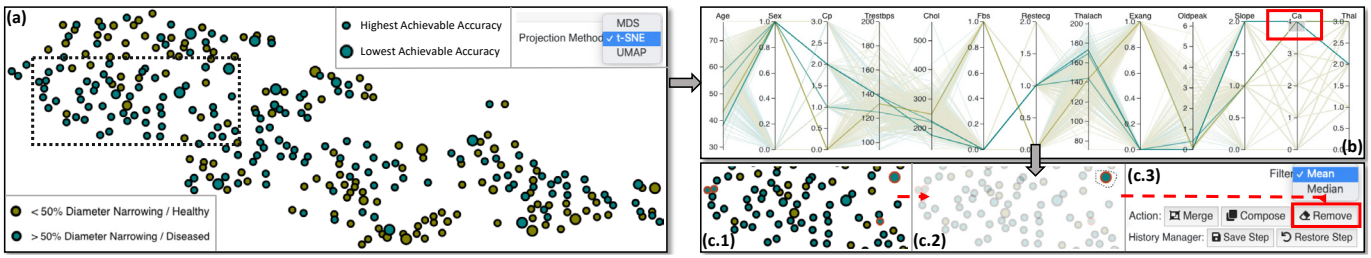

Fig. 4. The data space projection with the importance of each instance measured by the accuracy achieved by the stack models (a). The parallel coordinates plot view for the exploration of the values of the features (b); a problematic case is highlighted in red with values being null ('4' has no meaning for *Ca*). (c.1) shows the brushed instance from the selection in (b) and (c.2) a problematic point that causes troubles to the stacking ensemble. (c.3) indicates the various functionalities that StackGenVis is able to perform for instances.

(logistic regression (LR)), (f) two discriminant analysis classifiers (linear (LDA) and quadratic (QDA)), and (g) four ensemble classifiers (RF, extra trees (ExtraT), AdaB, and GradB).

In the following subsections, we explain the system by using a medicine data set, called *heart disease*, taken from the UCI Machine Learning repository [14]. The data set consists of 13 numerical features/attributes and 303 instances.

4.1 Data Sets and Performance Metrics

As mentioned in Sect. 1, the selection of the right performance metrics for different types of analytical problems and/or data sets is challenging. For example, a medical expert is usually very careful when it comes to handle false negative cases, since human lives may be at stake. In StackGenVis, we offer the option of using eight different metrics with distinct levels of contribution for each, depending on what is appropriate for each individual case. The available metrics are grouped into: threshold (→ *accuracy*, *g-mean*, *precision*, *recall*, *f-beta score*, and *MCC*); ranking (→ *ROC AUC*); and probability (→ *log loss*).

To illustrate how to choose different metrics (and with which weights), we start our exploration by selecting the *heart disease* data set in Fig. 1(a). Knowing that the data set is balanced, we pick accuracy (weight = 100%) instead of g-mean (weight = 0%), as seen in Fig. 1(a). The positive class (*diseased*) is more important than the cases that are healthy, so we use precision and recall instead of ROC AUC (0%). We also decide that the reproducibility of the results is slightly more important than simply reaching high precision, so we decrease the weight of precision to 80%. For the f-beta metric, the f2-score is chosen because false negative cases should be better monitored, since they are more important for the underlying problem. MCC is a combination of all f-beta scores and shows us both the false positive and false negative results, which is especially useful for comparing it with the f2-score. Log loss penalizes outliers, and in our case, we should be aware of outliers as we have sensitive healthcare data. Finally, four of the performance metrics include one more option—they are marked with an asterisk in Fig. 1(a)—to compute the individual *metric* based on *micro-, macro-, or weighted-average*. Micro-average aggregates the contributions of all classes to compute the average metric, whereas macro-average computes the *metric* independently for each class and then takes the average (therefore treating all classes equally). Weighted-average calculates the metrics for each label and finds their average weighted by support (the number of true instances for each label). The data set is a binary classification problem and contains 165 diseased and

138 healthy patients. Hence, we choose micro-average to weight the importance of the largest class, even though the impact is low because of the lack of any significant imbalance for the dependent variable. The dice glyphs visible on the right hand side of Fig. 1(a) are static and only used to indicate that specific views do not use all pre-selected metrics. For instance, the performance comparison view Fig. 1(c) only uses four metrics. After this initial tuning of the metrics, we press the *Confirm* button to move further to the exploration of algorithms.

4.2 Exploration of Algorithms

Fig. 3(a.1, a.2) presents the initial views of the 11 algorithms (and their models) currently implemented in StackGenVis. Fig. 3(a.1) uses boxplots to represent the performance of the currently unselected algorithms/models based on the metrics combination discussed previously. This compact visual representation provides an overview to users and allows them to decide which algorithms or specific models perform better based on statistical information. Fig. 3(a.2) displays overlapping barcharts for depicting the per-class performances for each algorithm, i.e., two colors for the two classes in our example. The more saturated bar in the center of each class bar represents the altered performance when the parameters of algorithms are modified. Note that the view only supports three performance metrics: precision, recall, and f1-score. The y-axes in both figures represent aggregated performance, while the different algorithms are arranged along the x-axis with different colors. Fig. 3(a.1) shows that KNN models perform well, but not all of them. We can click the KNN boxplot and further explore and tune the model parameters for KNN with an interactive parallel coordinates plot, as shown in Fig. 3(b), where six models are selected by filtering. Wang et al. [62] experimented with alternative visualization designs for selecting parameters, and they found that a parallel coordinates plot is a solid representation for this context as it is concise and also not rejected by the users. A drawback is the complexity of it compared to multiple simpler scatterplots. Fig. 3(c.1) indicates that, after the parameter tuning, the selected KNN models (narrow, more saturated bars) perform better than the average (wide, less saturated bars) and are thus good picks for our ensemble. Next, we perform similar steps for RF vs. ExtraT without class optimization as shown in Fig. 3(a.2, d).

Such iterative exploration proceeds for every algorithm until we are satisfied, see Fig. 3(e) where six algorithms are selected for our initial stack Ⓢ1. Fig. 3(f) shows a radar chart providing an overview of the entire space of available algorithms (yellow contour) against the current selection of models per algorithm (black star plot). In brackets, we show the number of all models for each algorithm, together with its name and representative color (Fig. 3(a.1, e)).

4.3 Data Wrangling

Pressing the *Execute Stacking Ensemble* button leads to the stacking ensemble shown in Fig. 1(b, Ⓢ1) with the performances shown at the end of the circular barcharts (in %) and in Fig. 1(c, Ⓢ1). Alternative designs we considered instead of the circular barcharts are standard barcharts or radial plots, but the labels of both would capture more vertical or horizontal space, respectively. In both panels, the performance of the metamodel is monitored with 4 out of the 8 metrics, which are accuracy, precision, recall, and f1-score. The line chart view is linked to the metrics of Fig. 1 with a dice glyph showing four. In Fig. 1(c), we encode the active stacking metrics with blue color and the stored stackings of Fig. 1(b, Ⓢ1–Ⓢ6) in red.

Data Space/Data Instances. Fig. 4(a) is a t-SNE projection [61] of the instances (MDS [22] and UMAP [31] are also available in order to empower the users with various perspectives for the same problem, based on the DR guidelines from Schneider et al. [47]). The point size is based on the predictive accuracy calculated using all the chosen models, with smaller size encoding higher accuracy value. Hence, we want to further investigate cases that cause problems (i.e., we have to look for large points). The parallel coordinates plot in Fig. 4(b) is used to investigate the features of the data set in detail.

The *Ca* attribute, for example, has a range of 0–3, but by selection we can see five points with *Ca* values of '4', see Fig. 4(b). These values can be considered as unknown and should be further examined. One of these points belongs to the *healthy* class (due to the olive color) but is very small in Fig. 4(c.1)—meaning that it does not reduce the accuracy. Four points are part of the *diseased* class. One of those is rather large which affects negatively the prediction accuracy of our classification (see Fig. 4(c.1) in the upper right corner). In Fig. 4(c.2), we select the point with our lasso interaction. We have then several options to manipulate this point as shown in Fig. 4(c.3): we can remove the point's instance entirely from the data set or merge a set of points into a new one, which receives either their mean or median values per feature. Similarly, we can compose a new point (i.e., an additional instance) from a set of points. The *history manager* saves the aforementioned manipulations or restores the previous saved step on demand. For our problematic point, we decide to remove it, and the metamodel performance increases as seen in Step 1 of Fig. 1(c) for the active model in blue. We then store this new stack and get the ensemble shown in Fig. 1(b, Ⓢ2) and Fig. 1(c, Ⓢ2). The details about the model's performance and parameters used can also be displayed with a tooltip.

Data Features. For the next stage of the workflow, we focus on the data features. Three different feature selection approaches can be used to compute the importance of each feature for each model in the stack. *Univariate* feature importance is identical for all models, but different for each feature. *Permutation* feature importance is measured by observing how random re-shuffling of each predictor influences model performance. *Accuracy* feature importance removes features one by one, similar to permutation, but then retrains each model by receiving only the accuracy as feedback. These last two approaches are very resource-intensive; therefore, they can be turned off for larger data sets (by disabling *Detailed Feature Search* in Fig. 1(a)). For our example in Fig. 5(a), they are enabled. We normalize the importance from 0 to 1 and use a two-hue color encoding from dark red to dark green to highlight the least to the most important features for our current stored stack, see Fig. 5(b). The panel in Fig. 5(c) uses a table heatmap view where data features are mapped to the y-axis (13 attributes, only 7 visible in the figure), and the x-axis represents the selected 204 models of stacking Ⓢ2. The available interactions for this view include panning and zooming in or out. Also, there is a possibility to check the *average value* of all models for each feature, serving as an overview. For our scenario, we can observe that *Trestbps, Chol, Fbs, and Restecg* are less important features. However, Fig. 5(c, right side) indicates that some models perform slightly better when including the *Chol* feature (due to the less saturated red color). Thus, we only disable the other three attributes by clicking the *Average* buttons in Fig. 5(c) on the right and get Fig. 5(d). After recalculating the performance of the active stacking metamodel (Step 2 of Fig. 1(c)), we store the improved stacking ensemble cf. Fig. 1(b, c, Ⓢ3).

4.4 Exploration of Models

The model exploration phase is perhaps the most important step on the way to build a good ensemble. It focuses on comparing and exploring different models both individually and in groups. Due to the page limits, we now assume that we selected the most performant models, removed the remaining from the stack, and reached Ⓢ4 (see Fig. 1(b)). Stack Ⓢ4 did not boost the performance due to the lack of diverse models from the KNN algorithm (cf. Fig. 1(c)). Diversity is one major component when building stack ensembles from scratch. The performance further drastically fell for Ⓢ5 (see Fig. 1(c)) when we reduced the number of models even more (marked as Step 3). As Step 3 led to bad results, we decided to go back to Ⓢ3 by clicking the *Stacking Ensemble 3* button in Fig. 1(b) to reactivate it.

Models' Space. For the visual exploration of the models shown in Fig. 6, we use MDS projections (t-SNE or UMAP are also available). Each point is one model from the stack, projected from an 8-dimensional space where each dimension of each model is the value of a user-weighted metric. Thus, groups of points represent clusters of models that perform similarly according to all the metrics. A summary of the performance of each model according to all selected and user-weighted metrics is color-encoded using the Viridis colormap [26]. The boxplots below the projection show the performance of the models per metric. Fig. 6(a) presents ensemble Ⓢ3, with all models still included.

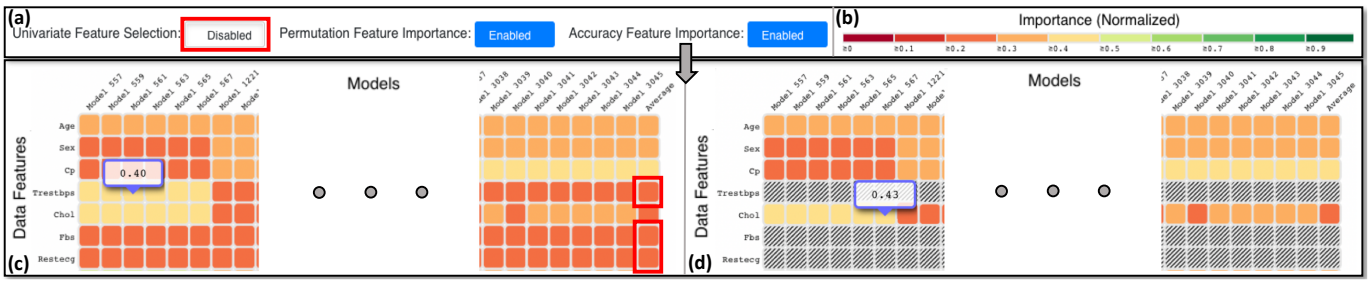

Fig. 5. Our feature selection view that provides three different feature selection techniques. The y-axis of the table heatmap depicts the data set's features, and the x-axis depicts the selected models in the current stored stack. Univariate-, permutation-, and accuracy-based feature selection is available as long with any combination of them (a). (b) displays the normalized importance color legend. The per-model feature accuracy is depicted in (c), and (d) presents the user's interaction to disable specific features to be used for all the models (only seven features are shown here). This could also happen on an individual basis for every model.

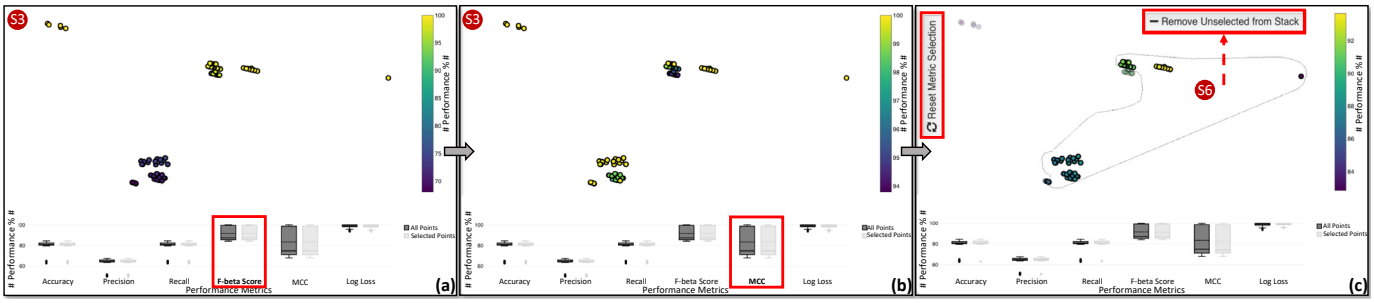

Fig. 6. Visual exploration of the models' space. The same MDS projection is observable in varying stages with different legend ranges and diverse colors for each instance, depending on the selected performance metric. The three steps in this figure demonstrate that we can reach both performant base models but also diverse algorithms by exploration of different validation metrics in (a) and (b). With the removal of the unselected models in (c), the performance remains stable but the complexity of the stacking ensemble reduces as more models leave the previous stack (cf. Fig. 1(b, Ⓢ6)).

Fig. 6(a+b) show the same projection but with different color-encodings for two selected performance metrics: f2-score and MCC. They allow us to decide which models are vital in order to stabilize the performance of the ensemble. For the f2-score (a), the complete cluster of models in dark blue (lower part) does not show good performance results; for MCC (b), the overall performance looks much better except for a small number of models in the center. To get rid of the most underperforming models and keep model diversity at the same time, we select, with the lasso tool, the best overall performing models under consideration of the worst performing models for f2-score and MCC (see Fig. 6(a+b)). We have now a new ensemble Ⓢ6 which presents the same results as Ⓢ3, but with 30 fewer models (from 204 to 174 based on six ML algorithms), see Step 4 in Fig. 1(b+c). As such, the complexity of the stacking ensemble has been reduced, and its training can be performed faster without the identified underperforming models. In Fig. 1(b, Ⓢ6), we also display the parent stack Ⓢ3 from which the final stack has been derived during the workflow.

Predictions' Space. The goal of the predictions' space visualization (Fig. 1(f)) is to show an overview of the performance of all models of the current stack for different instances. As in the data space, each point of the projection is an instance of the data set. However, instead of its original features, the instances are characterized as high-dimensional vectors where each dimension represents the prediction of one model. Thus, since there are currently 174 models in Ⓢ6, each instance is a 174-dimensional vector, projected into 2D. Groups of points represent instances that were consistently predicted to be in the same class. In Fig. 1(f), for example, the points in the two clusters in both extremes of the projection (left and right sides, unselected) are well-classified, since they were consistently determined to be in the same class by most models of Ⓢ6. The instances that are in-between these clusters, however, do not have a well-defined profile, since different models classified them differently. After selecting these instances with the lasso tool, the two histograms below the projection in Fig. 1(f) show a comparison of the performance of the available models in the selected points (gray, upside down) vs. all points (black). The x-axis represents the performance according to the user-weighted metrics (in bins of 5%), and the y-axis shows the number of models in each bin. Our goal here is to look for models in the current stack Ⓢ6 that could improve the performance for the selected points. However, by looking at the histograms, it does not look like we can achieve it this time, since all models perform worse in the selected points than in all points.

4.5 Results of the Metamodel

Recent work by Latha and Jeeva [24] tried out various ensembles for this same data set, such as bagging, boosting, stacking, and majority vote, combined with feature selection. They found that majority vote with the NB, BN, RF, and MLP algorithms was the best combination achieving 85.48% accuracy. For stacking, they reached ≈83% accuracy. With StackGenVis, we reached an accuracy of ≈88%, thus surpassing both of their ensembles. This shows that our VA approach can be effective when users combine base models to produce the best, most diverse, and simplest possible stacking ensemble. The results can be exported in the JSON format (Fig. 1(b, top-right), *Knowledge Extraction*), allowing users to apply the trained stacking ensemble with new data.

5 Use Case

In this section, we describe how StackGenVis can be used to improve the results of sentiment/stance detection in texts from social media, when compared to previous work from Skeppstedt et al. [51]. The authors studied the automatic detection of seven stance categories: *certainty, uncertainty, hypotheticality, prediction, recommendation, concession/contrast*, and *source*. Their model performed best for the *hypotheticality* category, using a baseline classification approach without the application of heavy feature selection/engineering, therefore we focus on this category in our comparison. It can be considered as a binary classification problem: the *presence* or *absence* of *hypotheticality*. The training data set was collected using the tool by Kucher et al. [23] and it consists of 2,095 instances of annotated training samples. The 300 feature vectors are based on the counts of the most frequent words in the corpus. The data set is very imbalanced, with most cases being

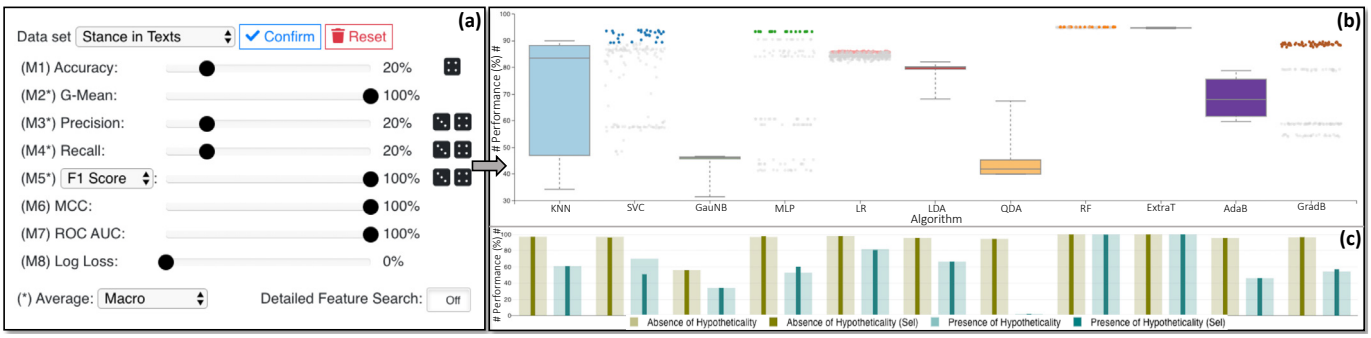

Fig. 7. The process of exploration of distinct algorithms in *hypotheticality* stance analysis. (a) presents the selection of appropriate validation metrics for the specification of the data set. (b) aggregates the information after the exploration of different models and shows the active ones which will be used for the stack in the next step. (c) presents the per-class performance of all the models vs. the active ones per algorithm.

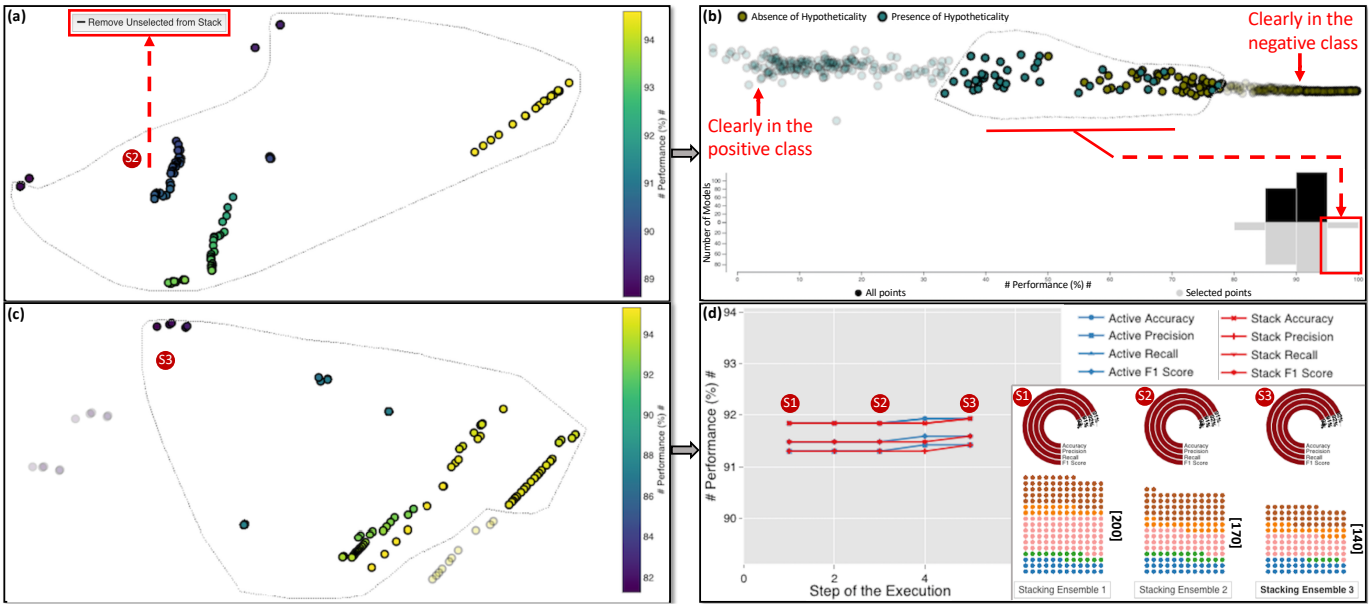

Fig. 8. The exploration of the models' and predictions' spaces and the metamodel's results. (a) presents the initial models' space and how it can be simplified with the removal of unnecessary models. The predictions' space is then updated, and the user is able to select instances that are not well classified by the stack of models in (b). This leads to an updated models' space in (c), where we can even fine-tune and choose diverse concrete models. The results of our actions can always be monitored in the performance line chart and the history preservation for stacks views in (d).

on the *absence* side. Skeppstedt et al. [51] used an SVM algorithm to train and build their baseline classifier for this task, and we are going to compare it to our stacking ensemble method in this use case.

Selection of Algorithms and Models. Similar to the workflow described in Sect. 4, we start by setting the most appropriate parameters for the problem (see Fig. 7(a)). As the data set is very imbalanced, we emphasize *g-mean* over *accuracy*, and *ROC AUC* over *precision* and *recall*. *Log loss* is disabled because the investigation of outliers is not critical for this text data set, and our computations do not have to be as precise as with medical data. Finally, due to the small number of instances in the presence of the *hypotheticality* class, we start with *macro-average*, which favors the smaller class (the previous work [51] did not explicitly discuss their averaging strategy). The resulting selection of algorithms can be seen in Fig. 7(b), where GradB is performing better than AdaB, and RF is slightly better than ExtraT. We improved the per-class performance (as shown in Fig. 7(c)) by choosing diverse ML models instead of simply the top-performing ones, since LR and RF perform well in the positive class, while other techniques such as SVC and GradB are far better in the negative class.

Optimized Models for Specific Predictions. In Fig. 8(a), we see the initial projection of the 200 models selected up to this point (i.e., ⓢ1). Some models perform well according to our metrics, but others could be removed due to lower performance. However, we should try not to break the balance between performance and diversity of our stacking ensemble. Thus, we choose to remove some of the models that are positioned close together and are not performing as expected (but not all of such models). The selection of ⓢ2 leads us to 170 models, cf. Fig. 8(d). By selecting these models, we get a new prediction space projection, shown in Fig. 8(b). While some predictions are clearly in the positive or negative class, we focus on the unclear cases and select them using the lasso tool. The updated histogram indicates in gray that there are better models available for the selected instances. Simultaneously, the models' space is updated as well, depicted in Fig. 8(c). Again, we try to preserve the diversity, but also reduce the complexity of the ensemble by removing the models with lower performance and low output diversity. As a result, in Fig. 8(d) we can see that the final stack ⓢ3 contains 140 models that perform better than the previous two stacks of 200 and 170 models. With 5-fold cross-validation, we reach 91%–92% performance for all our metamodel's validation metrics.

Evaluation of the Results with the Test Data Set. To confirm that our findings are solid, we applied the resulting metamodel to the same test data as Skeppstedt et al. [51], see Table 1. For the *hypotheticality* category, the reported f1-score for the baseline approach was 66%. In our case, we reached the following results with the final stack and *weighted-average*: 94.46% for accuracy, 93.87% for precision, 94.46% for recall, and f1-score of 93.87%. Additionally, as

an extra validation, we checked the results for the *prediction* category (again as a binary classification problem). Using our approach, we managed to achieve an f1-score of approximately 82% compared to 54% reported by Skeppstedt et al. [51] for the baseline approach. Finally, it is important to note that, while our approach seems to perform very well for both applications described in this paper, the gain does not come only from the performance. Our system supports the exploration and manipulation of many different perspectives of a complex stacking ensemble with the help of visualizations, which is the main burden for stacking ensemble learning to become even more broadly useful.

Table 1. Summary of the test data results for stance classification.

Performance Metric	StackGenVis		Skeppstedt et al. [51]	
	Hypotheticality	Prediction	Hypotheticality	Prediction
Accuracy	94.46%	88.21%	—	—
Precision	93.87%	77.81%	59%	51%
Recall	94.46%	88.21%	74%	57%
F1-score	93.87%	82.68%	66%	54%

6 Evaluation and Future Work

In this section we discuss the experts' feedback about StackGenVis, as well as possible improvements for our VA approach.

Methodology and Information about the Participants. Following guidelines from previous work [29, 34, 65], we conducted semi-structured interviews with three experts to gather qualitative feedback about the effectiveness and usefulness of our system. The first expert (**E1**) is a senior specialist in ML and analytics platforms working in a large multinational company. He has approximately 10 years of experience with ML. Moreover, at least half of his PhD studies (2.5 years) was specifically dedicated to stacking ensemble learning. The second expert (**E2**) is a senior researcher in software engineering and applied ML working in a government research institute and as an adjunct professor. He has worked with ML for the past 7 years, and 2 years with stacking ensemble learning. The third expert (**E3**) is the head of applied ML in a large multinational corporation, working with recommendation systems. She has approximately 7 years of experience with ML, of which 1.5 years are related to stacking ensemble learning. All three experts have a PhD in computer science and none of them reported any colorblindness issues. The process was as follows: (1) we presented the main goals of our system, (2) we explained the process of improving the *heart disease* data set results (see Sect. 4), and (3) after that, we gave them a couple of minutes to interact with the VA system by using the simple *iris* data set. During this process, we asked them to think aloud, as any feedback might be vital. However, to structure the process, we explained to them the basic components of our infrastructure that we would like to receive feedback upon. Each interview lasted about one hour, during which we recorded the screen and audio for further analysis. We summarize the key findings from the interviews below.

Workflow. **E1**, **E2**, and **E3** agreed that the workflow of StackGenVis made sense. They all suggested that data wrangling could happen before the algorithms' exploration, but also that it is usual to first train a few algorithms and then, based on their predictions, wrangle the data. Thus, it is considered an iterative process: the expert might start with the algorithms' exploration and move to the data wrangling, or vice versa. "The former approach is even more suitable for your VA system, because you use the accuracy of the base ML models as feedback/guidance to the expert in order to understand which instances should be wrangled", said **E3**. **E2** stated that having an evaluation metric from early on is important for benchmarking purposes to choose the best strategy while data scientists and domain experts are collaborating. He also noted that flexibility of the workflow—not forcing the user to use all parts of the VA system for every problem—is an extra benefit.

Visualization and Interaction. **E1** and **E3** were positively surprised by the power of visualization regarding the possibilities of dynamically and directly interacting with the ML algorithms and models. **E2** added that, after some initial training period (because the system could be a bit overwhelming in the beginning), the power of visualization in StackGenVis for supporting the analytical process is impressive. **E3** raised the question: "why not select the best, or a set of the best models of an algorithm, according to the performance, and why do we need visualization?" We answered that the per-class performance is also a very important component, and exploratory visualization can assist in the selection process, as seen in Fig. 3(b and c.1). The expert understood the importance of visualization in that situation, compared to not using it. Another positive opinion from **E3** was that, with a few adaptations to the performance metrics, StackGenVis could work with regression or even ranking problems. **E3** also mentioned that supporting *feature generation* in the feature selection phase might be helpful. Finally, **E1** suggested that the circular barcharts could only show the positive or negative difference compared to the first stored stack. To avoid an asymmetric design and retain a lower complexity level for StackGenVis, we omitted his proposal for the time being, but we consider implementing both methods in the future.

Limitations. *Efficiency and scalability* were the major concerns raised by **all the experts**. The inherent computational burden of stacking multiple models still remains, as such complex ensemble learning methods need sufficient resources. Also, the use of VA in between levels makes this even worse. We believe that, with the rapid development of high-performance hardware and support for parallelism, these challenges are due to diminish in the near future. Considering all that, **E3** noted that our system could be useful in solving competition problems, e.g., on Kaggle, and for her team to run tests before applying specific models to their huge data sets. Progressive VA workflows [53] could also be useful for improving the scalability of our approach for larger data sets. *Interpretability and explainability* is another challenge (mentioned by **E3**) in complicated ensemble methods, which is not necessarily always a problem depending on the data and the tasks. However, the utilization of user-selected weights for multiple validation metrics is one way towards interpreting and trusting the results of stacking ensembles. This is an advantage identified by **E2**. In the first use case we presented to him, he noted that: "if you are interested in the fairness of the results, you could show with the *history preservation view* of the system how you reached to these predictions without removing the *age* or *sex* features, consequently, not leading to discrimination against patients, for example". The visual exploration of stacking methods that use *multiple layers* [28] mentioned by **E1** is set as another future work goal. While the experts suggested that they almost never continue to stack models into more than one layer in their practice, we can investigate the adaptations for more layers required for our workflow. Finally, as this work was the first one working with stacking and visualization, we still need to investigate further the impact of *alternative metamodels* on the predictive performance (mentioned by **E1**) and try out *different modifications of stacking* [33], for instance, by adapting our workflow with an extra step of visually comparing various metamodels. It is in our plans to conduct a quantitative user study to further evaluate our system in the future.

7 Conclusion

In this paper, we introduced an interactive VA system, called StackGenVis, for the alignment of data, algorithms, and models in stacking ensemble learning. The adaptation of an already-existing knowledge generation model leads us to stable design goals and analytical tasks that were realized by StackGenVis. With the careful selection of multiple coordinated views, we allow users to build an effective stacking ensemble from scratch. Exploring the algorithms, the data, and the models from different perspectives and tracking the training process enables users to be sure how to proceed with the development of complex stacks of models that require a combination of not only the best performant but also the most diverse individual models. To retrieve preliminary results about the effectiveness of StackGenVis, we presented use cases with real-world data sets that demonstrated the improvements in performance and the process of achieving them. We also evaluated our approach with expert interviews by retrieving feedback about the workflow of our system, the interactive visualizations, and the limitations of our approach. Those limitations were then identified as future work for further development of our system.

REFERENCES

[1] M. Brehmer and T. Munzner. A multi-level typology of abstract visualization tasks. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2376–2385, Dec. 2013. doi: 10.1109/TVCG.2013.124

[2] L. Breiman. Random forests. *Machine Learning*, 45:5–32, Oct. 2001. doi: 10.1023/A:1010933404324

[3] A. Chatzimparmpas, R. M. Martins, I. Jusufi, and A. Kerren. A survey of surveys on the use of visualization for interpreting machine learning models. *Information Visualization*, 19(3):207–233, July 2020. doi: 10.1177/1473871620904671

[4] A. Chatzimparmpas, R. M. Martins, I. Jusufi, K. Kucher, F. Rossi, and A. Kerren. The state of the art in enhancing trust in machine learning models with the use of visualizations. *Computer Graphics Forum*, 39(3):713–756, June 2020. doi: 10.1111/cgf.14034

[5] A. Chatzimparmpas, R. M. Martins, and A. Kerren. t-viSNE: Interactive assessment and interpretation of t-SNE projections. *IEEE Transactions on Visualization and Computer Graphics*, 26(8):2696–2714, Aug. 2020. doi: 10.1109/TVCG.2020.2986996

[6] S. Chen, N. Andrienko, G. Andrienko, L. Adilova, J. Barlet, J. Kindermann, P. H. Nguyen, O. Thonnard, and C. Turkay. LDA ensembles for interactive exploration and categorization of behaviors. *IEEE Transactions on Visualization and Computer Graphics*, 2019. doi: 10.1109/TVCG.2019.2904069

[7] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pp. 785–794. ACM, 2016. doi: 10.1145/2939672.2939785

[8] D. Chicco and G. Jurman. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21:6, Jan. 2020. doi: 10.1186/s12864-019-6413-7

[9] J. F. Cohen, D. A. Korevaar, D. G. Altman, D. E. Bruns, C. A. Gatsonis, L. Hooft, L. Irwig, D. Levine, J. B. Reitsma, H. C. W. de Vet, and P. M. M. Bossuyt. STARD 2015 guidelines for reporting diagnostic accuracy studies: Explanation and elaboration. *BMJ Open*, 6:e012799, Nov. 2016. doi: 10.1136/bmjopen-2016-012799

[10] C. Collins, N. Andrienko, T. Schreck, J. Yang, J. Choo, U. Engelke, A. Jena, and T. Dwyer. Guidance in the human-machine analytics process. *Visual Informatics*, 2(3):166–180, Sept. 2018. doi: 10.1016/j.visinf.2018.09.003

[11] S. Das, D. Cashman, R. Chang, and A. Endert. BEAMES: Interactive multi-model steering, selection, and inspection for regression tasks. *IEEE Computer Graphics and Applications*, 39(9), Sept. 2019. doi: 10.1109/MCG.2019.2922592

[12] J. Davis and M. Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning*, ICML '06, pp. 233–240. ACM, 2006. doi: 10.1145/1143844.1143874

[13] E. Dimara, S. Franconeri, C. Plaisant, A. Bezerianos, and P. Dragicevic. A task-based taxonomy of cognitive biases for information visualization. *IEEE Transactions on Visualization and Computer Graphics*, 26(2):1413–1432, Feb. 2020. doi: 10.1109/TVCG.2018.2872577

[14] D. Dua and C. Graff. UCI machine learning repository. http://archive.ics.uci.edu/ml, 2017. Accessed April 23, 2020.

[15] C. Ferri, J. Hernández-Orallo, and R. Modroiu. An experimental comparison of performance measures for classification. *Pattern Recognition Letters*, 30(1):27–38, Jan. 2009. doi: 10.1016/j.patrec.2008.08.010

[16] Y. Freund, R. Schapire, and N. Abe. A short introduction to boosting. *Journal of Japanese Society for Artificial Intelligence*, 14(5):771–780, Sept. 1999.

[17] P. Isenberg, N. Elmqvist, J. Scholtz, D. Cernea, K.-L. Ma, and H. Hagen. Collaborative visualization: Definition, challenges, and research agenda. *Information Visualization*, 10(4):310–326, Oct. 2011. doi: 10.1177/1473871611412817

[18] L. Jonsson, M. Borg, D. Broman, K. Sandahl, S. Eldh, and P. Runeson. Automated bug assignment: Ensemble-based machine learning in large scale industrial contexts. *Empirical Software Engineering*, 21(4):1533–1578, Aug. 2016. doi: 10.1007/s10664-015-9401-9

[19] L. Jonsson, D. Broman, K. Sandahl, and S. Eldh. Towards automated anomaly report assignment in large complex systems using stacked generalization. In *Proceedings of the Fifth IEEE International Conference on Software Testing, Verification and Validation*, pp. 437–446. IEEE, 2012. doi: 10.1109/ICST.2012.124

[20] Kaggle Competition — Otto Group product classification challenge. https://kaggle.com/c/otto-group-product-classification-challenge, 2015. Accessed April 13, 2020.

[21] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS '17, pp. 3149–3157. Curran Associates Inc., 2017.

[22] J. B. Kruskal. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika*, 29(1):1–27, Mar. 1964. doi: 10.1007/BF02289565

[23] K. Kucher, C. Paradis, M. Sahlgren, and A. Kerren. Active learning and visual analytics for stance classification with ALVA. *ACM Transactions on Interactive Intelligent Systems*, 7(3), Oct. 2017. doi: 10.1145/3132169

[24] C. B. C. Latha and S. C. Jeeva. Improving the accuracy of prediction of heart disease risk based on ensemble classification techniques. *Informatics in Medicine Unlocked*, 16:100203, 2019. doi: 10.1016/j.imu.2019.100203

[25] S. Liu, J. Xiao, J. Liu, X. Wang, J. Wu, and J. Zhu. Visual diagnosis of tree boosting methods. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):163–173, Jan. 2018. doi: 10.1109/TVCG.2017.2744378

[26] Y. Liu and J. Heer. Somewhere over the rainbow: An empirical assessment of quantitative colormaps. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI '18, pp. 598:1–598:12. ACM, 2018. doi: 10.1145/3173574.3174172

[27] J. M. Lobo, A. Jiménez-Valverde, and R. Real. AUC: A misleading measure of the performance of predictive distribution models. *Global Ecology and Biogeography*, 17(2):145–151, Mar. 2008. doi: 10.1111/j.1466-8238.2007.00358.x

[28] R. Lorbieski and S. M. Nassar. Impact of an extra layer on the stacking algorithm for classification problems. *Journal of Computer Science*, 14(5):613–622, May 2018. doi: 10.3844/jcssp.2018.613.622

[29] Y. Ma, T. Xie, J. Li, and R. Maciejewski. Explaining vulnerabilities to adversarial machine learning through visual analytics. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):1075–1085, Jan. 2020. doi: 10.1109/TVCG.2019.2934631

[30] Z. Ma, P. Wang, Z. Gao, R. Wang, and K. Khalighi. Ensemble of machine learning algorithms using the stacked generalization approach to estimate the warfarin dose. *PLOS ONE*, 13(10):1–12, Oct. 2018. doi: 10.1371/journal.pone.0205872

[31] L. McInnes, J. Healy, and J. Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *ArXiv e-prints*, 1802.03426, Feb. 2018.

[32] S. M. McNee, J. Riedl, and J. A. Konstan. Being accurate is not enough: How accuracy metrics have hurt recommender systems. In *CHI '06 Extended Abstracts on Human Factors in Computing Systems*, CHI EA '06, pp. 1097–1101. ACM, 2006. doi: 10.1145/1125451.1125659

[33] E. Menahem, L. Rokach, and Y. Elovici. Troika — An improved stacking schema for classification tasks. *Information Sciences*, 179(24):4097–4122, Dec. 2009. doi: 10.1016/j.ins.2009.08.025

[34] Y. Ming, P. Xu, F. Cheng, H. Qu, and L. Ren. ProtoSteer: Steering deep sequence model with prototypes. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):238–248, Jan. 2020. doi: 10.1109/TVCG.2019.2934267

[35] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer. Formalizing visualization design knowledge as constraints: Actionable and extensible models in Draco. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):438–448, Jan. 2019. doi: 10.1109/TVCG.2018.2865240

[36] T. Mühlbacher and H. Piringer. A partition-based framework for building and validating regression models. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):1962–1971, Dec. 2013. doi: 10.1109/TVCG.2013.125

[37] S. Nagi and D. K. Bhattacharyya. Classification of microarray cancer data using ensemble approach. *Network Modeling Analysis in Health Informatics and Bioinformatics*, 2(3):159–173, 2013. doi: 10.1007/s13721-013-0034-x

[38] A. I. Naimi and L. B. Balzer. Stacked generalization: An introduction to super learning. *European Journal of Epidemiology*, 33(5):459–464, May 2018. doi: 10.1007/s10654-018-0390-z

[39] R. Nambiar Jyothi and G. Prakash. A deep learning-based stacked generalization method to design smart healthcare solution. In *Emerging Research in Electronics, Computer Science and Technology*, pp. 211–222. Springer

Singapore, 2019.

[40] W. Oliveira, L. M. Ambrósio, R. Braga, V. Ströele, J. M. David, and F. Campos. A framework for provenance analysis and visualization. *Procedia Computer Science*, 108:1592–1601, 2017. doi: 10.1016/j.procs.2017.05.216

[41] L. Pereira and N. Nunes. A comparison of performance metrics for event classification in non-intrusive load monitoring. In *Proceedings of the IEEE International Conference on Smart Grid Communications*, SmartGridComm '17, pp. 159–164. IEEE, 2017. doi: 10.1109/SmartGridComm.2017.8340682

[42] D. M. W. Powers. Evaluation: From precision, recall and F-measure to ROC, informedness, markedness & correlation. *Journal of Machine Learning Technologies*, 2(1):37–63, 2011.

[43] E. D. Ragan, A. Endert, J. Sanyal, and J. Chen. Characterizing provenance in visualization and data analysis: An organizational framework of provenance types and purposes. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):31–40, Jan. 2016. doi: 10.1109/TVCG.2015.2467551

[44] D. Sacha, A. Stoffel, F. Stoffel, B. C. Kwon, G. Ellis, and D. A. Keim. Knowledge generation model for visual analytics. *IEEE Transactions on Visualization and Computer Graphics*, 20(12):1604–1613, Dec. 2014. doi: 10.1109/TVCG.2014.2346481

[45] O. Sagi and L. Rokach. Ensemble learning: A survey. *WIREs Data Mining and Knowledge Discovery*, 8(4):e1249, July–Aug. 2018. doi: 10.1002/widm.1249

[46] T. Saito and M. Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, Mar. 2015. doi: 10.1371/journal.pone.0118432

[47] B. Schneider, D. Jäckle, F. Stoffel, A. Diehl, J. Fuchs, and D. A. Keim. Integrating data and model space in ensemble learning by visual analytics. *IEEE Transactions on Big Data*, 2018. doi: 10.1109/TBDATA.2018.2877350

[48] G. Sehgal, M. Rawat, B. Gupta, G. Gupta, G. Sharma, and G. Shroff. Visual predictive analytics using iFuseML. In *Proceedings of the EuroVis Workshop on Visual Analytics*, EuroVA '18. The Eurographics Association, 2018. doi: 10.2312/eurova.20181106

[49] B. Shneiderman. Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction*, 36(6):495–504, 2020. doi: 10.1080/10447318.2020.1741118

[50] G. Sigletos, G. Paliouras, C. D. Spyropoulos, and M. Hatzopoulos. Combining information extraction systems using voting and stacked generalization. *Journal of Machine Learning Research*, 6:1751–1782, Nov. 2005.

[51] M. Skeppstedt, V. Simaki, C. Paradis, and A. Kerren. Detection of stance and sentiment modifiers in political blogs. In *Speech and Computer*, vol. 10458 of *LNCS*, pp. 302–311. Springer International Publishing, 2017. doi: 10.1007/978-3-319-66429-3_29

[52] M. Sokolova and G. Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, July 2009. doi: 10.1016/j.ipm.2009.03.002

[53] C. D. Stolper, A. Perer, and D. Gotz. Progressive visual analytics: User-driven visual exploration of in-progress analytics. *IEEE Transactions on Visualization and Computer Graphics*, 20(12):1653–1662, Dec. 2014. doi: 10.1109/TVCG.2014.2346574

[54] B. L. Sturm. Classification accuracy is not enough. *Journal of Intelligent Information Systems*, 41(3):371–406, Dec. 2013. doi: 10.1007/s10844-013-0250-y

[55] J. Talbot, B. Lee, A. Kapoor, and D. S. Tan. EnsembleMatrix: Interactive visualization to support machine learning with multiple classifiers. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '09, pp. 1283–1292. ACM, 2009. doi: 10.1145/1518701.1518895

[56] A. Tharwat. Classification assessment methods. *Applied Computing and Informatics*, 2018. doi: 10.1016/j.aci.2018.08.003

[57] K. M. Ting and I. H. Witten. Stacked generalization: When does it work? In *Proceedings of the Fifteenth International Joint Conference on Artifical Intelligence — Volume 2*, IJCAI '97, pp. 866–871. Morgan Kaufmann Publishers Inc., 1997.

[58] C. Tong, R. Roberts, R. Borgo, S. Walton, R. S. Laramee, K. Wegba, A. Lu, Y. Wang, H. Qu, Q. Luo, and X. Ma. Storytelling and visualization: An extended survey. *Information*, 9(3):65, Mar. 2018. doi: 10.3390/info9030065

[59] J. Torres-Sospedra, C. Hernández-Espinosa, and M. Fernández-Redondo. Combining MF networks: A comparison among statistical methods and stacked generalization. In *Artificial Neural Networks in Pattern Recognition*, pp. 210–220. Springer Berlin Heidelberg, 2006. doi: 10.1007/11829898_19

[60] R. Tugay and Ş. Gündüz Öğüdücü. Demand prediction using machine learning methods and stacked generalization. In *Proceedings of the 6th International Conference on Data Science, Technology and Applications*, DATA '17, pp. 216–222. SciTePress, 2017. doi: 10.5220/0006431602160222

[61] L. van der Maaten and G. Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.

[62] Q. Wang, Y. Ming, Z. Jin, Q. Shen, D. Liu, M. J. Smith, K. Veeramachaneni, and H. Qu. ATMSeer: Increasing transparency and controllability in automated machine learning. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, pp. 681:1–681:12. ACM, 2019. doi: 10.1145/3290605.3300911

[63] D. H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992. doi: 10.1016/S0893-6080(05)80023-1

[64] K. Xu, S. Attfield, T. J. Jankun-Kelly, A. Wheat, P. H. Nguyen, and N. Selvaraj. Analytic provenance for sensemaking: A research agenda. *IEEE Computer Graphics and Applications*, 35(3):56–64, May–June 2015. doi: 10.1109/MCG.2015.50

[65] K. Xu, M. Xia, X. Mu, Y. Wang, and N. Cao. EnsembleLens: Ensemble-based visual exploration of anomaly detection algorithms with multidimensional data. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):109–119, Jan. 2019. doi: 10.1109/TVCG.2018.2864825

[66] J. Zhang, Y. Wang, P. Molino, L. Li, and D. S. Ebert. Manifold: A model-agnostic framework for interpretation and diagnosis of machine learning models. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):364–373, Jan. 2019. doi: 10.1109/TVCG.2018.2864499

[67] K. Zhao, M. O. Ward, E. A. Rundensteiner, and H. N. Higgins. LoVis: Local pattern visualization for model refinement. *Computer Graphics Forum*, 33(3):331–340, June 2014. doi: 10.1111/cgf.12389

[68] X. Zhao, Y. Wu, D. L. Lee, and W. Cui. iForest: Interpreting random forests via visual analytics. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):407–416, Jan. 2019. doi: 10.1109/TVCG.2018.2864475